

Seentair Limited — Full Project Documentation

Client: Seentair Limited | Contact: Okereke Ndubuka Cynthia

Approved 9th September 2026

Engineering Architecture & Design Revision ? 15 September 2026

Table of Contents

Seentair Limited — Full Project Documentation

- How this package is organized
- Important note on technical decisions
- Suggested reading order for a new team member

01. Project Bible — Seentair Limited Platform

- What this document is
- Client
- The one-sentence vision
- The guiding principle (north star)
- Core goals
- What this platform explicitly is NOT (for now)
- Glossary
- Key stakeholders
- Related documents

02. Product Requirements Document (PRD)

- 1. Purpose & Scope
- 2. In-scope modules (MVP — “Must Have Now”)
- 3. Deferred modules (“Important Later”)
- 4. Functional requirements by module
- 5. Non-functional requirements
- 6. Out of scope for v1
- 7. Source of truth

03. Open Questions / Final Clarification List

- Process

04. Product Architecture

- System overview (mermaid)
- Applications / surfaces
- Why one central system
- Core domains inside the Central Business System
- Integration boundary
- Related documents

05. Role & Permission Matrix

- Rules to enforce in the permission system
- Open item

06. UX/UI Requirements

- Guiding principle
- Reference point: Temu (for order tracking only)
- Design principles
- Key screen inventory (MVP)
- Notification touchpoints
- Accessibility & localization

07. Technical Architecture

- Confirmed stack
- System diagram
- Architectural principles (non-negotiable, derived from client requirements)
- Security requirements
- Environments

08. Initial Data Model

- Entity-relationship overview (mermaid)
- Core entities
- Notes for the engineer implementing this

09. API Specification (v1 — high level)

- Conventions
- Auth
- Users & Roles
- Products & Catalogue
- Manufacturing & Production
- Raw Materials
- Inventory
- Orders (retail, wholesale, in-store all share this resource, filtered by channel)
- Wholesale
- Custom Orders (post-MVP)
- Returns
- Reviews
- Accounting

- Logistics
- Analytics
- Partners (post-MVP)
- Approvals & Audit
- Notifications

10. Development Guide

- Repository structure (proposed monorepo)
- Branching strategy
- Commit convention
- Code review requirements
- Coding standards
- Local environment setup (outline)
- Definition of Done (per ticket)

11. QA & Testing Strategy

- Testing levels
- Critical scenarios requiring E2E coverage
- Test data & environments
- UAT process (recommended)
- Regression protection

12. Operations & Deployment Guide

- Environments
- CI/CD (proposed)
- Monitoring & alerting
- Backups
- Logging
- Incident response (outline)
- Data protection

13. Project Plan / Development Phases

- Phase 0 — Foundation (Sprint 0)
- Phase 1 — Core Data Foundation
- Phase 2 — Manufacturing
- Phase 3 — Retail Commerce
- Phase 4 — Wholesale
- Phase 5 — Operations
- Phase 6 — Governance & Insight
- Phase 7 — Engagement (Post-MVP, “Important Later”)
- Cadence recommendation
- Dependency notes

14. Scope (scope.json)

15. Design Specification

Appendix: Requirements

- How to use this package
- Contents

01. Business Understanding

- Company
- Core business
- Strategic goals
- Platform framing
- Confirmed by client

02. Manufacturing & Production

- Production model
- Production stages (confirmed, adjustable to real factory floor terms)
- Cost tracking
- Rejects / QC failures
- Data/entities implied

03. Raw Materials

- What should still be tracked
- Data/entities implied

04. Products

- What needs to be managed per product
- Data/entities implied
- Confirmed by client

05. Retail

- Customer-facing shopping flow
- Notes
- Data/entities implied

06. Wholesale & Bulk Buyers

- Eligibility

- Capabilities for wholesale/bulk buyers
 - Data/entities implied
07. Custom / Pre-Order & Special Design Requests
- Confirmed end-to-end flow
 - Information required from the buyer at submission
 - Approval & pricing
 - Data/entities implied
08. Orders & Payments
- Rules
 - Data/entities implied
09. Order Tracking
- Reference point
 - Confirmed customer-facing journey
 - Notifications
 - Data/entities implied
10. Returns
- What must be recorded per return
 - Confirmed policy details (from clarification round)
 - Data/entities implied
11. Inventory
- Why it matters
 - Required visibility
 - Key requirement: movement history, not just a number
 - Data/entities implied
12. Accounting & Finance
- Purpose
 - Areas requiring visibility
 - Reporting requirement (from clarification round)
 - Credit policy
 - Data/entities implied
13. Suppliers
- What to build instead
 - Note for the team
14. Logistics & Delivery
- Delivery methods in use
 - Pricing
 - Integration priority
 - Requirements
 - Data/entities implied
15. Marketing & Sales Growth
- Business objective
 - Capabilities to support
 - Data/entities implied
16. Analytics & Management Dashboard
- Goal
 - Dashboard data points
 - Data/entities implied
17. Partners / Investors
- Important clarification
 - Their role and access
 - Confirmed dashboard flow
 - Profit-sharing model (confirmed in clarification round)
 - Data/entities implied
 - Priority note
18. Staff & Access Control
- Scale
 - Principle
 - Confirmed roles (illustrative, not necessarily exhaustive)
 - Requirement
 - Data/entities implied
19. Approvals & Audit Trail
- Requirement
 - Actions confirmed to require approval/authorization
 - Data/entities implied
20. Communication
- Principle

- Channels
- Data/entities implied

21. Integrations

- Confirmed/identified integration points
- Data migration

22. POS / Mall Sales System

- Vision
- Expected connections
- Status
- Priority conflict — needs resolution

23. Future Vision

- Explicitly NOT part of the initial platform direction
- Longer-term direction (post-MVP)
- The single most important outcome (guiding principle)

24. MVP Priorities

- Must Have Now
- Important Later
- △ Contradiction to resolve before sprint planning
- Suggested MVP build order (proposed, pending client sign-off)

25. Remaining Clarification Questions

26. Final Approval / Sign-off

- Signed
- What happens next

Seentair Limited — Full Project Documentation

This package is the complete documentation set for commencing development of the Seentair fashion manufacturing & commerce platform, prepared after client sign-off on 9th September 2026.

How this package is organized

#	Document	Purpose
01	Project Bible	Single source of truth — vision, goals, principles, glossary
02	PRD	Full functional requirements, module by module
03	Open Questions / Final Clarification List	Unresolved items to close before/during Sprint 0
04	Product Architecture	How the modules and user-facing apps fit together
05	Role & Permission Matrix	Who can do what
06	UX/UI Requirements	Design principles, key flows, screen inventory
08	Initial Data Model	Entities, fields, relationships
09	API Specification	Endpoint list by module
10	Development Guide	Repo structure, branching, coding standards
11	QA & Testing Strategy	Test levels, critical scenarios, UAT process
12	Operations & Deployment Guide	Environments, CI/CD, monitoring, backups
13	Project Plan / Development Phases	Sprint-by-sprint delivery plan
14	scope.json	Machine-readable summary of the above, for tooling
Appendix	Requirements (26 files)	The approved, module-by-module requirements this is all built on

Important note on technical decisions

Cynthia (the client) approved the business requirements ? what the system must do. The engineering team has now confirmed the implementation stack for Sprint 0: Angular for every user-interaction frontend, Node.js + NestJS for the backend API, and PostgreSQL for the central database. Hosting and certain external providers remain team/budget decisions where marked TBD. The new Design Specification defines the public storefront?s cinematic scroll-driven fashion direction.

Suggested reading order for a new team member

- 01-Project-Bible.md — get oriented
- 02-PRD.md — understand what’s being built
- 08-Initial-Data-Model.md + 04-Product-Architecture.md — understand the shape of the system
- 13-Project-Plan-Development-Phases.md — understand the delivery order
- Everything else, as needed for your role

01. Project Bible — Seentair Limited Platform

What this document is

The single, always-current reference for what we are building and why. When in doubt, this is the tie-breaker. Detailed specs live in the other documents; this one holds the *intent*.

Client

- **Business:** Seentair Limited
- **Contact:** Okereke Ndubuka Cynthia
- **Industry:** Fashion manufacturing (streetwear/clothing), one factory
- **Customers served:** general public (retail), retailers, wholesalers, distributors/bulk buyers — children, teens, and adults

The one-sentence vision

A central business system connecting manufacturing, inventory, sales (retail + wholesale), e-commerce, accounting, logistics, analytics, and partner/investor visibility — replacing manual, paper-based operations with one connected digital platform.

The guiding principle (north star)

“Customers can shop without struggling, while the owner can see what is happening in the business without needing to be physically present at the factory.”

Every product and design decision should be checked against this sentence. If a feature doesn’t make shopping easier for customers *or* make the business more visible to its owner, question why it’s being built.

Core goals

1. Move the business off manual/paper record-keeping onto one digital system.
2. Give the owner real-time visibility (“a heartbeat”) into the business without being physically present.
3. Increase sales and make purchasing easier for customers.
4. Give business partners/investors full transparency into their investment performance.
5. Reduce wastage and loss through better tracking and calculation.

What this platform explicitly is NOT (for now)

- Not a marketplace connecting unrelated third-party sellers.
- Not an affiliate/referral/commission platform — “partners” means business investors, not marketing affiliates.
- Not a multi-factory system (single factory for the foreseeable future).
- Not migrating any historical manual records — starting fresh.

Glossary

Term	Meaning in this project
Partner / Investor	A person who has invested money in the business and receives quarterly profit share. Not an affiliate.
MOQ	Minimum Order Quantity for wholesale eligibility — confirmed as 20 units.
Batch	A production run of a given product/style, tracked through manufacturing stages.
Custom/Pre-order	A buyer-submitted design request, quoted manually, requiring sample approval before full production.
Central system	The single backend all sales channels (online, in-store, wholesale) must feed into — no channel operates in isolation.

Key stakeholders

- **Client / final approver:** Okereke Ndubuka Cynthia (Seentair Limited)
- **Internal roles the system must support:** Business Owner/Admin, Sales, Inventory, Production, Finance/Accounting, Management, Partner/Investor, Customer (see 05-Role-Permission-Matrix.md)

Related documents

See 00-README.md for the full document map.

02. Product Requirements Document (PRD)

1. Purpose & Scope

This PRD consolidates the approved requirements for the Seentair platform into one functional specification, module by module. Each module below summarizes confirmed business rules; full detail and the client's exact confirmed language lives in `Appendix-Requirements/`.

2. In-scope modules (MVP — “Must Have Now”)

Manufacturing & Production, Raw Materials, Product Catalogue, Inventory, Retail E-commerce, Wholesale, Payments, Order Tracking, Returns, Accounting, Logistics, Marketing, Analytics, Staff & Access Management, Approvals & Audit Trail, WhatsApp/Live Chat.

3. Deferred modules (“Important Later”)

Custom/Pre-order requests, Partner/Investor portal, SMS notifications. (POS/physical sales has a scheduling contradiction — see `03-Open-Questions.md` item 1 — tentatively treated as deferred.)

4. Functional requirements by module

4.1 Manufacturing & Production

- Production runs in batches; quantity driven by previous sales + current orders.
- Stages: Production Planned → Cutting → Sewing → Finishing → Quality Control → Completed (customizable to real factory terms).
- Production cost = raw material + sewing + branding + packaging cost.
- Rejects: defective → burned; minor factory error → repaired & restocked. Reason code is required and drives disposition.

4.2 Raw Materials

- No supplier-relationship management (client sources from multiple, non-fixed suppliers — see descoping note in Appendix `13-Suppliers.md`).
- Track: materials, purchases (qty + cost), usage tied to production, available quantity, low-stock alerts.

4.3 Products

- Product, category, size, colour, collection, SKU, price, image(s), availability, linked production batch.

4.4 Retail

- Flow: discover → view → select variant → cart → checkout → pay online (Paystack) → confirmation → track → post-delivery review.
- Reviews are a client-requested addition, tied to delivered orders.

4.5 Wholesale

- MOQ = 20 units. Multiple wholesale price tiers exist (criteria TBD — see Open Questions).
- Portal: wholesale pricing, bulk orders, reorder, order/invoice history, tracking, custom design submission, reviews.

4.6 Custom / Pre-Order (deferred to post-MVP)

- Flow: submit design → business review → feasibility/cost → quotation → buyer accepts → payment → payment confirmation → sample production → buyer approves sample → full production → fulfilment → delivery.
- Required intake fields: size, colour, quantity, location, fabric quality, description, desired date.
- Design approval: Manager role. Quotation: manual (fabric, quantity, style-based).

4.7 Payments

- No part-payments on ordinary orders — full payment required upfront.
- Online: Paystack. Offline: cash/POS/bank transfer, recorded manually.
- All channels feed into one central order/payment record.

4.8 Order Tracking

- Customer-facing states: Order Received → Processing/Packaging → Shipped/Waybill → At Customer Location/Delivered → Returned (if applicable) → optional Review.
- SMS specifically for delivery/location updates (deferred integration — see 4.9 Notifications priority).

4.9 Returns

- Request within 12 hours of receipt; return completed within 24 hours.
- All products returnable except custom/special orders.
- Returned via logistics company with tracking/parcel ID.
- Must record: order, product, reason, disposition (restocked/damaged), resolution.

4.10 Inventory

- Full visibility: raw materials, WIP, finished goods, available/used/ sold/damaged/returned/dispatched stock.
- **Architectural requirement:** maintain a movement history/ledger, not a single mutable quantity field (see `08-Initial-Data-Model.md`).

4.11 Accounting

- Replaces manual accounting entirely.
- Tracks: sales, expenses, purchases, production cost, COGS, payroll, taxes, profit, loss.
- Full report suite required: income, expenditure, investment, loss, profit.
- No customer credit — customers are not expected to owe the business.

4.12 Logistics

- Dispatch riders (local), logistics companies, transport companies, interstate and international delivery.
- Delivery price = weight + location. Multi-leg deliveries supported.
- First integration: GIGL.

4.13 Marketing

- Campaigns, promotions, channel attribution, recommendations, loyalty, product visibility, performance analysis.

4.14 Analytics

- Dashboard: sales, revenue, profit/loss, best/slow sellers, low stock, production status, inventory status, product profitability, sales by channel, marketing source performance, partner info (where permitted).

4.15 Partners/Investors (deferred to post-MVP)

- Business investors, not affiliates. No access to customer-facing storefronts.
- Dashboard flow: Login → Business Overview → Investment Info → Performance → Inventory Visibility → Accounts/Reports → Profit Sharing.
- Profit split: 40% reinvestment / 40% dividends / 20% reserve. 1,000,000 total shares: 60% founder/CEO, 40% partners by equity %.

4.16 Staff & Access Control

- ~10 internal users. Roles: Business Owner/Admin, Sales, Inventory, Production, Finance/Accounting, Management, Partner/Investor, Customer.
- Role-based permissions; sensitive actions restricted.

4.17 Approvals & Audit Trail

- Approval required for: purchasing, production, price changes, moving funds, and any unauthorized material/product removal, transfer, or disposal.
- Full audit log: actor, action, before/after state, timestamp.

4.18 Communication

- In-platform messages, SMS, WhatsApp, live chat — centered on orders and delivery.

4.19 Integrations

- Paystack, WhatsApp, live chat, SMS, GIGL (logistics), marketing/social channels, future POS/mall systems. No historical data migration.

4.20 POS / Mall Sales System (scheduling TBD — see Open Questions)

- Connects to inventory, products, orders, payments, customer info, accounting, analytics. Exact screens/hardware deferred to a later planning stage regardless of priority-level outcome.

5. Non-functional requirements

- **Auditability:** all stock and fund movements must be traceable to an actor and timestamp (see 4.10, 4.17).
- **Data integrity over convenience:** inventory must never be a freely-editable number; it must be derived from logged movements.
- **Mobile-friendly:** retail and wholesale storefronts must work well on mobile, given WhatsApp/social-driven customer behavior already in use.
- **Fresh start:** no legacy data migration required for v1.

6. Out of scope for v1

Multiple factories, third-party marketplace, affiliate/commission system, full POS hardware integration (pending Open Question resolution), historical data migration.

7. Source of truth

Every rule above is sourced from `Appendix-Requirements/`. Where this PRD and the appendix ever disagree, treat the appendix as authoritative and flag the discrepancy for correction.

03. Open Questions / Final Clarification List

These do not block starting architecture or data-model work, but should be resolved with the client before their related module enters development.

#	Question	Affects	Priority
1	POS/physical sales: client’s direct answer said “LATER,” but the feature-priority table marked it “Must Have Now.” Which is correct?	Sprint planning, 13-Project-Plan-Development-Phases.md	High — resolve before Sprint 0
2	What criteria assign a wholesale customer to a price tier (volume, relationship length, manual negotiation)?	06-Role-Permission-Matrix.md , wholesale pricing logic	Medium
3	Is the sample-production cost in custom orders included in the original quotation, or billed separately?	Custom order payment flow	Medium
4	Should product reviews be published immediately or moderated/approved first?	Retail/Wholesale review feature	Medium
5	Partner/Investor portal is marked “Important Later,” but its profit-sharing model is already fully specified — worth pulling forward?	13-Project-Plan-Development-Phases.md	Low
6	Confirm currency (assumed ₦ Naira) given mentions of international delivery.	Payments, Accounting, Data Model	High — confirm before payment integration
7	Does NDPR (Nigeria Data Protection Regulation) compliance need to be built in from day one, given customer PII will be stored?	07-Technical-Architecture.md , 11-QA-Testing-Strategy.md	High — confirm before data model finalization

Process

Log the client’s answer to each item directly in this file (with date), and update the corresponding spec document. Do not let these sit unresolved past Sprint 0 planning for the High-priority items.

04. Product Architecture

System overview (mermaid)



Applications / surfaces

1. **Retail Storefront** (public-facing web, mobile-friendly) Discover → cart → checkout → track → review.
2. **Wholesale Portal** (authenticated) Wholesale pricing, bulk ordering, invoices, custom design submission.
3. **Admin / Operations Dashboard** (internal, role-restricted) Manufacturing, inventory, orders, accounting, analytics, staff, approvals — the primary tool for Business Owner/Admin, Sales, Inventory, Production, Finance, and Management roles.
4. **Partner/Investor Portal** (deferred) Read-mostly view: business overview, investment info, performance, inventory visibility, accounts/reports, profit sharing.
5. **POS / Mall Sales System** (deferred, scope TBD)

Why one central system

All four surfaces above must read/write through the same core business system — the client was explicit that no sales channel (online, in-store, wholesale) should operate in isolation. This is the single most important architectural constraint: **there is one source of truth for inventory, orders, and money**, not four separate silos with reconciliation later.

Core domains inside the Central Business System

- Catalogue (Products, Variants, Collections)
- Manufacturing (Batches, Costs, QC/Rejections)
- Inventory (Movement ledger)
- Sales (Orders, Payments, Returns) — spans retail + wholesale + custom
- Accounting (Ledger, Reports)
- Logistics (Deliveries, Tracking)
- People (Users, Roles, Partners, Customers)
- Governance (Approvals, Audit Trail)
- Engagement (Marketing, Notifications, Reviews)

Integration boundary

External services (Paystack, GIGL, SMS/WhatsApp providers) are accessed through dedicated integration modules/adapters within the core system — never called directly from the storefront or portal frontends. This keeps API keys server-side and makes it possible to swap providers later (e.g. adding a second logistics company beyond GIGL) without touching frontend code.

Related documents

`07-Technical-Architecture.md` (how this is actually built), `08-Initial-Data-Model.md` (the data behind each domain).

05. Role & Permission Matrix

Legend: **F** = Full access, **V** = View only, **A** = Can approve, **O** = Own records only, - = No access

Module ↓ / Role →	Business Owner/Admin	Management	Sales	Inventory	Production	Finance/Accounting	Partner/Investor	Wholesaler	Retail Customer
Manufacturing & Production	F	V	-	V	F	V	V (summary)	-	-
Raw Materials	F	V	-	F	V	V	-	-	-
Products/Catalogue	F	F	V	V	V	-	V	V	V
Inventory	F	V	V	F	V	V	V (summary)	-	-
Retail Orders	F	V	F	V	-	V	-	-	O
Wholesale Orders	F	V	F	V	-	V	-	O	-
Custom/Pre-order	F	A	F	-	V	-	-	O	-
Payments	F	V	V	-	-	F	V (summary)	O	O
Returns	F	A	F	V	-	V	-	O	O
Accounting/Finance	F	V	-	-	-	F	V (permitted reports)	-	-
Logistics	F	V	V	V	-	-	-	O	O (own delivery)
Marketing	F	F	V	-	-	-	-	-	-
Analytics/Dashboard	F	F	V (own area)	V (own area)	V (own area)	V (own area)	V (permitted)	-	-
Partners/Investors module	F	V	-	-	-	V	O	-	-
Staff & Access Control	F	V	-	-	-	-	-	-	-
Approvals & Audit Trail	F	A	-	-	-	A (finance items)	-	-	-
Communication tools	F	V	F (customer msgs)	-	-	-	-	O	O

Rules to enforce in the permission system

- **Approval gating:** purchasing, production starts, price changes, and fund transfers require an explicit approval action from Management or Business Owner/Admin — regular staff cannot self-approve.
- **No silent overrides:** any action affecting inventory or funds must write to the audit trail regardless of role.
- **Partner/Investor isolation:** this role must never have access to the retail/wholesale customer-facing interfaces or raw customer PII — client was explicit about this boundary.
- **Least privilege by default:** new roles/staff accounts start with View-only access to a module until explicitly granted Full access.

Open item

Exact approval chain (who approves what — Manager vs Business Owner/Admin) for each action type should be confirmed against [Appendix-Requirements/19-Approvals-Audit-Trail.md](#) during technical design; the table above is a reasonable default based on confirmed requirements, not a client-specified matrix.

06. UX/UI Requirements

Guiding principle

“Customers can shop without struggling, while the owner can see what is happening in the business without needing to be physically present.”

Every screen should be evaluated against this: does it reduce friction for a customer, or increase visibility for the owner? If it does neither, question its priority.

Reference point: Temu (for order tracking only)

The client referenced Temu specifically for the **order-tracking experience** — clear, real-time visibility into where an order is — not the overall Temu shopping experience or aesthetic. Don’t over-index on Temu-style discounts/gamification unless separately requested.

Design principles

1. **Mobile-first.** Customers currently interact via WhatsApp and social media; the retail and wholesale experiences must work comfortably on a phone.
2. **Low-friction checkout.** Full payment upfront (no part-payments), so checkout should be a short, clear, single pass — not a multi-step form.
3. **Status always visible.** Order status (Received → Processing → Shipped → Delivered → Returned) should be visible without the customer needing to ask.
4. **Role-appropriate dashboards.** Internal admin screens should surface only what that role needs (see 05-Role-Permission-Matrix.md) — a warehouse worker’s screen should look different from an accountant’s.
5. **One-glance owner dashboard.** The Business Owner/Admin’s home screen should answer, at a glance: sales today, profit/loss, low-stock alerts, production status, and anything pending approval.

Key screen inventory (MVP)

Retail Storefront

- Product discovery/browse & search
- Product detail (variant selection: size/colour)
- Cart & checkout (Paystack)
- Order confirmation
- Order tracking page
- Post-delivery review submission
- Return request form

Wholesale Portal

- Wholesale product catalogue with tiered pricing
- Bulk order form / reorder shortcut
- Order & invoice history
- Order tracking
- (Post-MVP) Custom design submission form

Admin / Operations Dashboard

- Owner home dashboard (sales, profit/loss, alerts, approvals pending)
- Production board (batches by stage — Kanban-style fits well here)
- Raw material inventory & low-stock alerts
- Product catalogue management
- Order management (all channels, filterable)
- Returns queue
- Accounting/reports view
- Staff & role management
- Approval queue (pending purchases, price changes, fund movements)
- Audit log viewer

Deferred (Post-MVP)

- Partner/Investor dashboard (per confirmed flow in Appendix-Requirements/17-Partners-Investors.md)
- Custom/pre-order management screens
- POS/mall interface

Notification touchpoints

- Order status changes → in-platform + SMS (delivery-specific) / WhatsApp (post-MVP for SMS specifically, per priority table)
- Return request approved/rejected → in-platform + customer’s preferred channel
- Pending approval → in-platform alert to Management/Owner

Accessibility & localization

- Currency formatting pending confirmation (see 03-Open-Questions.md #6) — build currency as a configurable value, not hardcoded.
- Support common Nigerian mobile network conditions: design for low-bandwidth resilience (compressed images, lightweight pages).

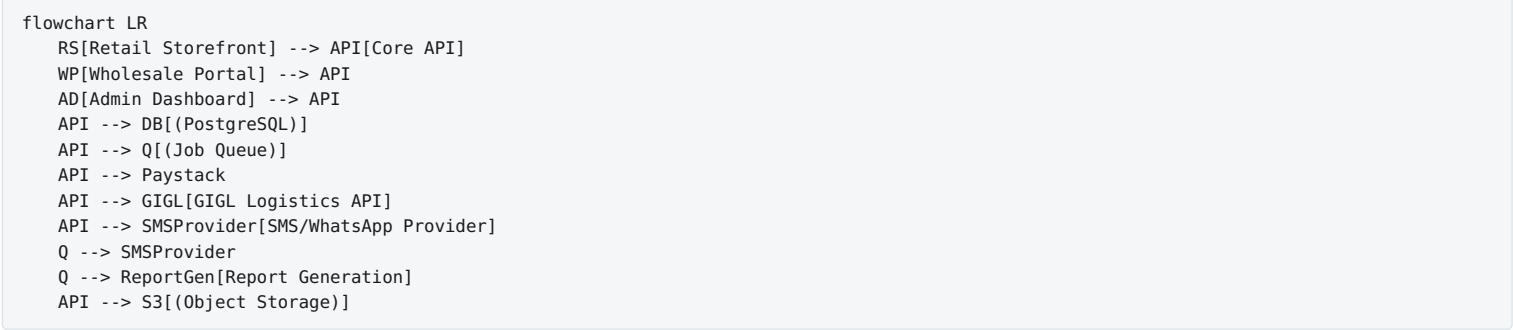
07. Technical Architecture



Confirmed engineering stack

		domains
Database	PostgreSQL	Relational integrity needed for inventory/accounting; strong audit/ledger support
File/image storage	S3-compatible object storage (AWS S3 or DigitalOcean Spaces)	Product images, review photos
Auth	JWT-based session + role-based access control (RBAC)	Matches 05-Role-Permission-Matrix.md
Payments	Paystack (confirmed)	Client-specified
SMS/WhatsApp	Termii or Twilio (TBD)	Local Nigerian delivery reliability — confirm provider
Logistics	GIGL API integration first (confirmed), designed as a pluggable adapter for future carriers	Client-specified
Hosting	AWS or DigitalOcean (TBD by team/budget)	Either works for this scale
Background jobs	A queue (e.g. BullMQ on Redis) for notifications, report generation	Keeps API responsive

System diagram



Architectural principles (non-negotiable, derived from client requirements)

- Single source of truth.** One Core API and one database serve all frontends — no channel-specific data silos.
- Inventory is event-sourced.** Never mutate a stock count directly; always write an `InventoryMovement` record and derive the current count. This directly satisfies the client’s explicit requirement for reliable movement history.
- Money-moving and stock-moving actions are approval-gated at the API layer,** not just hidden in the UI — a request to change price or move funds without proper approval state should be rejected server-side.
- Everything writes to the audit log.** Implement as a database trigger or service-layer interceptor, not something each feature remembers to call manually.
- Currency and locale are configuration, not hardcoded** (pending Open Question #6).

Security requirements

- All customer PII (name, phone, location, order history) encrypted at rest; access logged.
- Role-based access control enforced server-side on every endpoint, not just hidden in the frontend.
- Payment credentials (Paystack keys) never exposed to frontend code.
- Pending Open Question #7: confirm NDPR compliance requirements before finalizing data retention and export policies.

Environments

Development → Staging → Production (see `12-Operations-Deployment-Guide.md` for details).

08. Initial Data Model

This is a first-pass entity list derived from the approved requirements. Expect refinement during technical design, but this should be a solid starting point for the database schema.

Entity-relationship overview (mermaid)



Core entities

User & Access

- **User:** id, name, phone, email, role_id, status
- **Role:** id, name (Business Owner/Admin, Sales, Inventory, Production, Finance/Accounting, Management, Partner/Investor, Customer)
- **Permission:** role_id, module, access_level (F/V/A/O/-)

Catalogue

- **Product:** id, name, description, category, base_price
- **ProductVariant:** id, product_id, size, colour, SKU, price_override, image_url, availability_status
- **Collection:** id, name

Manufacturing

- **ProductionBatch:** id, product_id, quantity, stage, planned_date, completed_date
- **ProductionCost:** id, batch_id, material_cost, sewing_cost, branding_cost, packaging_cost
- **QCRejection:** id, batch_id, reason, disposition (burned | repaired_restocked), date

Raw Materials

- **RawMaterial:** id, name, unit, current_quantity, reorder_threshold
- **MaterialPurchase:** id, material_id, date, quantity, cost, note
- **MaterialUsage:** id, material_id, batch_id, quantity_used, date

Inventory

- **InventoryMovement:** id, variant_or_material_id, type (purchase | production | sale | return | damage | dispatch | adjustment), quantity_delta, actor_id, timestamp, reference_id

Sales

- **Order:** id, customer_id, channel (retail | wholesale | custom | in_store), status, payment_status, created_at
- **OrderItem:** id, order_id, variant_id, quantity, unit_price
- **Payment:** id, order_id, method (paystack | cash | pos | bank_transfer), amount, status, recorded_by, date
- **DeliveryLeg:** id, order_id, carrier, leg_number, status, tracking_ref
- **ReturnRequest:** id, order_id, variant_id, reason, requested_at, return_deadline, tracking_number, resolution, restocked (bool), damaged (bool)
- **Review:** id, order_id, variant_id, customer_id, rating, comment, created_at, status (pending | published) — pending Open Question #4

Wholesale

- **WholesaleAccount:** id, user_id, tier_id, approved_moq_status
- **PriceTier:** id, name, rule_description — pending Open Question #2

Custom Orders (post-MVP)

- **CustomOrderRequest:** id, buyer_id, sizes, colours, quantity, location, fabric_quality, description, desired_date, status
- **Quotation:** id, request_id, amount, approved_by, created_at
- **SampleApproval:** id, request_id, sample_status, buyer_approved (bool), date

Accounting

- **LedgerEntry:** id, type (sale | expense | purchase | payroll | tax), amount, category, date, reference_id
- **FinancialReport:** id, type, period, generated_data

Partners (post-MVP)

- **Partner:** id, user_id, equity_percentage, invested_amount
- **ProfitDistribution:** id, period, total_profit, reinvestment_amount, dividend_pool, reserve_amount, per_partner_breakdown (JSON)

Governance

- **ApprovalRequest:** id, action_type, requested_by, status, approved_by, decided_at
- **AuditLogEntry:** id, actor_id, action, before_state (JSON), after_state (JSON), timestamp

Engagement

- **Notification:** id, recipient_id, channel, type, related_order_id, sent_at
- **Campaign:** id, name, type, start_date, end_date

Notes for the engineer implementing this

- `InventoryMovement` is the source of truth for stock — `current_quantity` fields elsewhere should be derived/cached, not authoritative.
- `AuditLogEntry` should be written automatically (trigger or interceptor), not manually per-feature, to avoid gaps in coverage.
- Currency fields should reference a configurable currency setting (pending Open Question #6), not assume a single hardcoded currency.

09. API Specification (v1 — high level)

This is a high-level REST endpoint inventory to guide backend scaffolding. A full OpenAPI/Swagger spec should be generated from this during technical design once the framework (see 07-Technical-Architecture.md) is confirmed.

Conventions

- Base path: /api/v1
- Auth: Bearer JWT on all endpoints except /auth/* and public storefront read endpoints (GET /products , etc.)
- All list endpoints support pagination (?page=&limit=)
- All mutating endpoints that touch inventory or funds must resolve through the approval/audit layer described in 07-Technical-Architecture.md

Auth

- POST /auth/login
- POST /auth/logout
- POST /auth/refresh

Users & Roles

- GET /users (F: Admin)
- POST /users (F: Admin)
- GET /users/:id
- PATCH /users/:id/role (F: Admin)

Products & Catalogue

- GET /products (public)
- GET /products/:id (public)
- POST /products (Admin/Management)
- PATCH /products/:id (Admin/Management — price changes require approval)
- GET /products/:id/variants
- POST /products/:id/variants

Manufacturing & Production

- GET /production-batches
- POST /production-batches
- PATCH /production-batches/:id/stage
- POST /production-batches/:id/cost
- POST /production-batches/:id/qc-rejection

Raw Materials

- GET /materials
- POST /materials
- POST /materials/:id/purchase
- POST /materials/:id/usage
- GET /materials/low-stock

Inventory

- GET /inventory/:variantId/movements
- POST /inventory/:variantId/movements (system-generated in most cases — sale, return, production completion — rather than manual direct calls)
- GET /inventory/summary

Orders (retail, wholesale, in-store all share this resource, filtered by channel)

- POST /orders
- GET /orders
- GET /orders/:id
- PATCH /orders/:id/status
- POST /orders/:id/payment
- GET /orders/:id/tracking

Wholesale

- GET /wholesale/pricing
- POST /wholesale/accounts

- GET /wholesale/accounts/:id

Custom Orders (post-MVP)

- POST /custom-orders
- GET /custom-orders/:id
- POST /custom-orders/:id/quotation
- POST /custom-orders/:id/sample-approval

Returns

- POST /returns
- GET /returns
- PATCH /returns/:id/resolve

Reviews

- POST /orders/:id/review
- GET /products/:id/reviews
- PATCH /reviews/:id/status (moderation — pending Open Question #4)

Accounting

- GET /accounting/ledger
- GET /accounting/reports/:type (income | expenditure | investment | loss | profit)

Logistics

- POST /deliveries
- PATCH /deliveries/:id/status
- GET /deliveries/:id/tracking

Analytics

- GET /analytics/dashboard
- GET /analytics/best-sellers
- GET /analytics/low-stock

Partners (post-MVP)

- GET /partners/:id/dashboard
- GET /partners/:id/profit-distributions

Approvals & Audit

- GET /approvals/pending
- POST /approvals/:id/decide
- GET /audit-log

Notifications

- POST /notifications/send (internal use, triggered by system events)

10. Development Guide

Engineering stack is confirmed: all user-interaction frontends use Angular; the backend uses Node.js + NestJS.

Repository structure (confirmed monorepo direction)

```
/seentair-platform
/apps
  /storefront      (Angular retail storefront)
  /wholesale-portal (Angular, authenticated)
  /admin-dashboard (Angular internal operations app)
/services
  /api             (core backend ? Node.js + NestJS)
  /jobs            (background workers)
/packages
  /shared-types    (shared TypeScript contracts)
  /ui-components   (shared design-system components)
/docs              (this documentation set)
```

Branching strategy

- `main` — always deployable, maps to production.
- `develop` — integration branch, maps to staging.
- `feature/<ticket-id>-short-description` — one branch per task, merged into `develop` via pull request.
- `hotfix/<ticket-id>` — branched from `main` for urgent production fixes.

Commit convention

Conventional Commits (`feat:` , `fix:` , `chore:` , `refactor:` , `test:` , `docs:`), so changelogs can be generated automatically.

Code review requirements

- Minimum 1 approving review before merge to `develop` .
- Any change touching inventory movement, payments, or the approval/audit layer requires review from a second engineer familiar with that domain — these are the highest-risk areas per `07-Technical-Architecture.md` .
- CI must pass (lint, type-check, unit tests) before merge is allowed.

Coding standards

- TypeScript strict mode across frontend and backend.
- Linting: ESLint + Prettier, shared config in `/packages` .
- No direct database writes to stock/quantity fields from feature code — always go through the inventory movement service layer (see `08-Initial-Data-Model.md` notes).
- Environment variables never committed; use `.env.example` as the template.

Local environment setup (outline)

1. Clone repo, install dependencies per app/service.
2. Copy `.env.example` → `.env` in each app/service, fill in local Paystack test keys, DB connection string.
3. Run database migrations.
4. Seed minimal reference data (roles, a test product, a test user per role) for local development.
5. Start services (`api` , then frontends).

Definition of Done (per ticket)

- Meets the acceptance criteria in the ticket (sourced from `02-PRD.md` / `Appendix-Requirements/`).
- Unit tests added for new logic.
- No new lint/type errors.
- Reviewed and approved per the rules above.
- Deployed to staging and smoke-tested before being marked done.

11. QA & Testing Strategy

Testing levels

1. **Unit tests** — business logic in isolation (e.g. production cost calculation, MOQ eligibility check, profit-distribution math).
2. **Integration tests** — API endpoints against a real test database, including permission enforcement per `05-Role-Permission-Matrix.md`.
3. **End-to-end (E2E) tests** — critical user journeys through the UI (see below).
4. **User Acceptance Testing (UAT)** — with the client, before each major release, mirroring the same YES/CHANGE/NOT SURE review pattern already used successfully during requirements sign-off.

Critical scenarios requiring E2E coverage

- **Full retail purchase:** browse → cart → checkout → Paystack payment → order confirmation → status updates → delivery → review.
- **Wholesale bulk order:** MOQ enforcement, tiered pricing, invoice generation.
- **Inventory integrity:** a sale correctly decrements available stock via a logged movement, never a direct field edit; a return correctly restocks (or flags as damaged) via the same mechanism.
- **Production → inventory flow:** completing a production batch correctly increases finished-goods inventory.
- **Approval gating:** attempting a price change, fund movement, or purchase without approval must be rejected by the API, not just hidden in the UI.
- **Audit trail completeness:** every stock/fund-affecting action produces a corresponding audit log entry — test this as a cross-cutting assertion, not per-feature.
- **Returns window enforcement:** request logged within 12 hours of delivery is accepted; after 12 hours is rejected (pending exact enforcement rule confirmation).
- **Role boundary:** Partner/Investor role cannot reach retail/wholesale customer-facing endpoints or raw customer PII.

Test data & environments

- Staging should use anonymized or synthetic data — no real customer PII copied from production once real data exists.
- Paystack test/sandbox keys in all non-production environments.

UAT process (recommended)

Reuse the same client review format that worked for requirements sign-off: for each release candidate, provide the client a short list of what was built, ask for YES / CHANGE / NOT SURE per feature, and log her answers the same way `Appendix-Requirements/` was produced. This keeps her engagement pattern consistent across the whole project.

Regression protection

Any bug found in production involving inventory, payments, or the audit trail must get a regression test added before the fix is merged — these are the domains where the client has been most explicit about correctness (movement history, no unauthorized fund/stock changes).

12. Operations & Deployment Guide

Proposed defaults — confirm hosting provider and budget with client/ team before committing.

Environments

Environment	Purpose	Data
Development	Local/engineer machines	Synthetic seed data
Staging	Pre-production testing, UAT with client	Synthetic/anonymized data, Paystack sandbox
Production	Live system	Real data, Paystack live keys

CI/CD (proposed)

- On pull request: lint, type-check, unit + integration tests.
- On merge to `develop`: auto-deploy to Staging.
- On merge to `main` (via release PR from `develop`): auto-deploy to Production after manual approval gate.

Monitoring & alerting

- Application error tracking (e.g. Sentry or similar).
- Uptime monitoring on the Core API and all customer-facing storefronts.
- Alert the team on: payment webhook failures (Paystack), failed inventory movement writes, and any spike in approval-request rejections (may indicate a permissions bug).

Backups

- Daily automated database backups, retained on a rolling window (e.g. 30 days), with at least weekly backups retained longer-term.
- Backup restore should be tested periodically, not just taken on faith.

Logging

- Centralized application logs, retained long enough to support the audit-trail requirement (`19-Approvals-Audit-Trail.md`) — the audit log itself lives in the database as a first-class record, separate from ephemeral application logs.

Incident response (outline)

- Detect (monitoring alert or user report).
- Triage severity — payment or inventory-affecting incidents are highest priority given the client’s explicit emphasis on correctness in these areas.
- Communicate status to the client if customer-facing.
- Fix, deploy, and add a regression test per `11-QA-Testing-Strategy.md`.
- Brief post-incident note in the project log.

Data protection

Pending Open Question #7 (NDPR compliance) — until confirmed, default to conservative practice: encrypt PII at rest, log all access to customer records, and avoid exporting raw customer data outside the production environment.

13. Project Plan / Development Phases

This plan sequences MVP delivery based on `Appendix-Requirements/24-MVP-Priorities.md` and dependency order (you can't track inventory before you have products; you can't take payments before you have orders, etc.).

Phase 0 — Foundation (Sprint 0)

- Resolve High-priority Open Questions (#1 POS priority, #6 currency, #7 NDPR) — see `03-Open-Questions.md`.
- Confirm technical stack (`07-Technical-Architecture.md`).
- Set up repo, CI/CD, environments (`10`, `12`).
- Implement Users/Roles/Permissions foundation.

Phase 1 — Core Data Foundation

- Product catalogue & variants.
- Raw materials tracking.
- Inventory movement ledger (the architectural backbone — build this right the first time, per client's explicit requirement).

Phase 2 — Manufacturing

- Production batches & stage tracking.
- Production cost tracking.
- QC rejection handling (burn vs. repair-and-restock).

Phase 3 — Retail Commerce

- Storefront: browse, cart, checkout.
- Paystack integration.
- Order tracking (customer-facing states).
- Post-delivery reviews.

Phase 4 — Wholesale

- Wholesale portal & tiered pricing (pending Open Question #2).
- Bulk ordering, reorder, invoice history.

Phase 5 — Operations

- Returns workflow (12hr/24hr windows).
- Accounting ledger & full report suite.
- Logistics integration (GIGL) & delivery pricing.

Phase 6 — Governance & Insight

- Approvals workflow (purchasing, production, price, funds).
- Audit trail (should already exist as a cross-cutting concern from Phase 0, but full UI/reporting here).
- Analytics dashboard.
- Marketing tools.

Phase 7 — Engagement (Post-MVP, “Important Later”)

- WhatsApp/Live chat.
- SMS notifications.
- Custom/pre-order request flow.
- Partner/Investor portal.
- POS/mall sales system (pending Open Question #1 resolution).

Cadence recommendation

2-week sprints, with a client review/UAT checkpoint (per `11-QA-Testing-Strategy.md`) at the end of Phases 3, 5, and 6 at minimum — these are the points where the client will most want to see and confirm real functionality.

Dependency notes

- Phase 1 blocks almost everything else — prioritize getting inventory ledger design right before building on top of it.
- Phase 3 (Payments) blocks Phase 4 (Wholesale uses the same payment path) and Phase 5 (Returns reference payments).
- Phase 6 (Approvals/Audit) should have its *foundation* built in Phase 0, even though full UI comes later — retrofitting audit logging after the fact is much more error-prone.

14. Scope (scope.json)

```
{
  "project": "Seentair Limited Platform",
  "client": {
    "business_name": "Seentair Limited",
    "contact_name": "Okereke Ndubuka Cynthia",
    "approved_date": "2026-09-09"
  },
  "guiding_principle": "Customers can shop without struggling, while the owner can see what is happening in the business without needing be physically present at the factory.",
  "modules": {
    "must_have_now": [
      "manufacturing",
      "production_tracking",
      "raw_materials",
      "inventory",
      "product_catalogue",
      "retail_ecommerce",
      "wholesale",
      "payments",
      "order_tracking",
      "returns",
      "accounting",
      "logistics",
      "marketing",
      "analytics",
      "staff_management",
      "approvals_audit_trail",
      "whatsapp_live_chat"
    ],
    "important_later": [
      "custom_pre_order_requests",
      "partner_investor_portal",
      "sms_notifications"
    ],
    "contradictory_status": [
      {
        "module": "pos_mall_sales_system",
        "direct_answer": "later",
        "feature_table_answer": "must_have_now",
        "resolution_status": "unresolved",
        "see": "03-Open-Questions.md#1"
      }
    ],
    "explicitly_out_of_scope": [
      "multiple_factories",
      "third_party_marketplace",
      "affiliate_commission_model",
      "historical_data_migration"
    ]
  },
  "roles": [
    "business_owner_admin",
    "sales",
    "inventory",
    "production",
    "finance_accounting",
    "management",
    "partner_investor",
    "wholesaler",
    "retail_customer",
    "distributor"
  ],
  "confirmed_business_rules": {
    "wholesale_moq": 20,
    "wholesale_price_tiers_exist": true,
    "wholesale_price_tier_criteria": "unresolved - see Open Question #2",
    "return_request_window_hours": 12,
    "return_completion_window_hours": 24,
    "returns_excluded": [
      "custom_orders",
      "special_requests"
    ],
    "part_payments_allowed_ordinary_orders": false,
    "online_payment_provider": "paystack",
    "first_logistics_integration": "GIGL",
    "delivery_pricing_basis": [
      "weight",
      "location"
    ],
    "production_cost_components": [
      "raw_material_cost",
```

```

        "sewing_cost",
        "branding_cost",
        "packaging_cost"
    ],
    "qc_rejection_dispositions": [
        "burned",
        "repaired_restocked"
    ],
    "profit_split": {
        "reinvestment_pct": 40,
        "dividends_pct": 40,
        "reserve_pct": 20
    },
    "total_shares": 1000000,
    "share_allocation": {
        "founder_ceo_pct": 60,
        "partners_pct": 40
    },
    "data_migration_required": false,
    "customer_credit_allowed": false,
    "supplier_relationship_management_required": false
},
"integrations": [
    "paystack",
    "whatsapp",
    "live_chat",
    "sms",
    "gigl_logistics",
    "marketing_social_channels",
    "future_pos_mall_systems"
],
"open_questions_count": 7,
"open_questions_high_priority": [
    "pos_priority_contradiction",
    "currency_confirmation",
    "ndpr_compliance_requirement"
],
"documentation_files": [
    "00-README.md",
    "01-Project-Bible.md",
    "02-PRD.md",
    "03-Open-Questions.md",
    "04-Product-Architecture.md",
    "05-Role-Permission-Matrix.md",
    "06-UX-UI-Requirements.md",
    "07-Technical-Architecture.md",
    "08-Initial-Data-Model.md",
    "09-API-Specification.md",
    "10-Development-Guide.md",
    "11-QA-Testing-Strategy.md",
    "12-Operations-Deployment-Guide.md",
    "13-Project-Plan-Development-Phases.md",
    "14-scope.json"
]
}

```


Seentair Limited — Development Kickoff Documentation

Client: Seentair Limited Client contact: Okereke Ndubuka Cynthia Approved: 9th September 2026 (see 25-Final-Approval-Signoff.md)

This package contains the approved, development-ready specification for the Seentair fashion manufacturing & commerce platform, broken into one file per functional area. It reflects the client’s confirmed answers from the Client Requirements Review & Approval document, plus corrections/additions she made during that review.

How to use this package

- Each numbered file below is a self-contained brief for that functional area: what’s confirmed, the business rules to build against, and any data/entities implied.
- 24-MVP-Priorities.md tells you what must ship first.
- 25-Remaining-Clarification-Questions.md lists a small number of unresolved items (including two direct contradictions in the client’s own answers) that should be confirmed with her before or during Sprint 0.
- 26-Final-Approval-Signoff.md is the record of client sign-off authorizing development to begin.

Contents

1. Business Understanding
2. Manufacturing & Production
3. Raw Materials
4. Products
5. Retail
6. Wholesale
7. Custom / Pre-Order Requests
8. Payments
9. Order Tracking
10. Returns
11. Inventory
12. Accounting
13. Suppliers
14. Logistics
15. Marketing
16. Analytics
17. Partners / Investors
18. Staff & Permissions
19. Approvals & Audit Trail
20. Communication
21. Integrations
22. POS / Mall Sales System
23. Future Vision
24. MVP Priorities
25. Remaining Clarification Questions
26. Final Approval / Sign-off

01. Business Understanding

Company

Seentair Limited — a fashion manufacturing factory operating from a single factory location.

Core business

- Focus: streetwear and clothing.
- Serves children, teens, and adults.
- Sells to: the general public (retail), retailers, wholesalers, and distributors/bulk buyers.

Strategic goals

- Grow beyond the current physical-only operation.
- Build a strong online clothing store.
- Increase sales and make purchasing easier for customers.
- Give the owner visibility into the business without needing to be physically present at the factory (“a heartbeat of the business”).
- Give business partners/investors transparency into how their funds are performing — explicitly, the client wants partners to “not feel cheated in any way.”

Platform framing

This is **not** just an online store. It is a central business system connecting manufacturing, inventory, sales (retail + wholesale), e-commerce, accounting, logistics, analytics, and partner/investor visibility, all in one place.

Confirmed by client

YES, this is an accurate description of the business (with the partner transparency note above added).

02. Manufacturing & Production

Production model

- Production runs in **batches**.
- Batch quantity is influenced mainly by **previous sales** and **current orders**.

Production stages (confirmed, adjustable to real factory floor terms)

Production Planned → Cutting → Sewing → Finishing → Quality Control → Completed

The system should let staff see which stage each batch is currently at.

Cost tracking

Production cost = **raw material cost + sewing cost + branding cost + packaging cost**. The system should let these four cost components be recorded/estimated per product or per batch so real production cost is visible to management.

Rejects / QC failures

Handling depends on the reason for rejection: - **Defective / bad product** → burned (write-off, removed from stock permanently). - **Simple factory error** (e.g. cosmetic, fixable) → repaired and put back into stock.

The system should capture the *rejection reason* as a required field, since it determines which of the two paths above is taken, and should track disposition (burned vs. repaired-and-restocked) for reporting.

Data/entities implied

- ProductionBatch (product, quantity, stage, planned date, completed date)
- ProductionCost (batch or product id, material cost, sewing cost, branding cost, packaging cost)
- QCRejection (batch/unit reference, reason, disposition: burned | repaired_restocked)

03. Raw Materials

Scope correction from client: during review, the client clarified that her earlier mention of “suppliers” referred to *the people/businesses she supplies to* (i.e. her customers), not raw-material suppliers. She does **not** have a stable material supplier and sources from multiple ones interchangeably. As a result, **no dedicated supplier-management module is needed for raw materials** — see 13-Suppliers.md for the corresponding scope note on that section.

What should still be tracked

Materials themselves (not supplier identity): - Record materials (name, type, unit of measure). - Record material purchases (date, quantity, cost — supplier name not required as a structured field, can be a free-text note if useful). - Track quantities on hand. - Track material usage (tied to production where appropriate). - Track material cost (feeds into production cost — see 02-Manufacturing-Production.md). - Show available material at any time. - Low-stock warnings/alerts.

Data/entities implied

- RawMaterial (name, unit, current quantity, reorder threshold)
- MaterialPurchase (material id, date, quantity, cost, optional note)
- MaterialUsage (material id, production batch id, quantity used, date)

04. Products

What needs to be managed per product

- Product (name, description)
- Category
- Size
- Colour
- Collection
- SKU / stock identifier
- Price (retail and wholesale price — see 06-Wholesale.md for tiers)
- Product image(s)
- Availability (in stock / out of stock / made-to-order)
- Link to production batch(es)

Data/entities implied

- Product (name, description, category, base price)
- ProductVariant (product id, size, colour, SKU, price override, image, availability)
- Collection (name, list of product ids)

Confirmed by client

YES, this represents how products should be managed.

05. Retail

Customer-facing shopping flow

Discover products → View product info → Select size/colour/variant → Add to cart → Checkout → Pay online → Receive confirmation → Track order → Receive updates → Request return (if applicable) → Leave a review (added by client)

Notes

- The retail experience should be simple and should encourage more purchases.
- **Reviews:** client explicitly added that customers should be able to leave a review after receiving their product(s). Build this as a post-delivery review flow, tied to order/product.
- Online payment provider: **Paystack** (see 08-Payments.md).

Data/entities implied

- `Review` (order id, product id, customer id, rating, comment, created_at)

06. Wholesale & Bulk Buyers

Eligibility

- A customer becomes a wholesale/bulk buyer by being able to meet the **minimum order quantity (MOQ)**.
- **Confirmed MOQ: 20 units.**
- There are **multiple wholesale price levels/tiers** (client confirmed “Yes” — exact tier structure/criteria is not yet defined, see 25-Remaining-Clarification-Questions.md).

Capabilities for wholesale/bulk buyers

- Access wholesale products/prices.
- Place bulk orders.
- Reorder products.
- View order information.
- Track orders.
- View relevant invoices/payment information.
- Submit special/custom design requests (see 07-Custom-Pre-Order.md).
- Receive quotations where necessary.
- Leave a review (client added this here too, same as retail).

Data/entities implied

- WholesaleAccount (customer id, tier, approved MOQ status)
- PriceTier (tier name, discount or fixed price rules) — pending exact tier definition

07. Custom / Pre-Order & Special Design Requests

Confirmed end-to-end flow

Buyer submits request/design

→ Business reviews

→ Business determines feasibility/cost

→ Quotation issued

→ Buyer accepts quotation

→ Buyer makes payment

→ Payment confirmation

→ Sample production

→ Buyer approves sample (gate before full production)

→ Production

→ Fulfilment

→ Delivery

Note: the sample-production and buyer-approval steps were added by the client during review — full production must not begin until the buyer has approved the sample.

Information required from the buyer at submission

- Size(s)
- Colour(s)
- Quantity
- Delivery location
- Fabric quality
- Name/description of the design
- Desired delivery date/timeline

Approval & pricing

- **Design approval authority:** Manager.
- **Quotation method:** manual, based on fabric quality/type, quantity, and style/sewing complexity. (No automated pricing formula yet — build for a manual quote-entry step.)

Data/entities implied

- CustomOrderRequest (buyer id, sizes, colours, quantity, location, fabric quality, description, desired date, status)
- Quotation (request id, amount, approved_by, created_at)
- SampleApproval (request id, sample status, buyer approved y/n, date)

08. Orders & Payments

Rules

- **No part-payments for ordinary orders.** Orders must be fully paid before proceeding through the applicable sales process. (Note: custom/pre-order flow is the exception — it has its own payment-then-sample-then-production sequence, see 07-Custom-Pre-Order.md.)
- **Online payments:** Paystack.
- **Offline sales:** recorded manually into the system. Offline payment methods include cash, POS, and bank transfer.
- All sales channels (online + offline/in-store) must ultimately feed into the **central business system** — no channel should operate in isolation.

Data/entities implied

- `Payment` (order id, method: paystack | cash | pos | bank_transfer, amount, status, recorded_by [for offline], date)
- `Order` (channel: online | in-store | wholesale | custom, payment status, fulfillment status)

09. Order Tracking

Reference point

Client cited Temu specifically as inspiration for the *order-tracking experience* — not to copy Temu generally, but to give customers the same level of clear visibility into their own order status.

Confirmed customer-facing journey

Order Received → Processing/Packaging → Shipped/Waybill →
At Customer Location/Delivered → Returned (if applicable) →
Review (optional, added by client)

Notifications

- Customers should be able to see order status themselves in the platform at any time.
- Important updates communicated through the platform.
- **SMS** specifically for delivery/location-related updates.

Data/entities implied

- OrderStatusEvent (order id, status, timestamp)
- Ties into Review entity from 05-Retail.md (optional post-delivery review)

10. Returns

What must be recorded per return

- Which order was returned
- Which product was returned
- Why it was returned (reason)
- What happened to the returned item
- Whether it goes back into inventory
- Whether it is damaged/rejected
- What resolution was given

Confirmed policy details (from clarification round)

- **Return request window:** must be raised within **12 hours** of receiving the order.
- **Return completion window:** the physical return should happen within **24 hours** of the request.
- **Eligibility:** any product can be returned **except special/custom design orders** (see `07-Custom-Pre-Order.md` — these are excluded since they're made-to-spec).
- **Return logistics:** the item is sent back via a logistics company, and a tracking number/parcel ID is shared with the business.

Data/entities implied

- `ReturnRequest` (order id, product id, reason, requested_at, return_deadline, tracking_number, resolution, restocked: bool, damaged: bool)

11. Inventory

Why it matters

Client confirmed inventory is one of the most important parts of the system.

Required visibility

- Raw materials
- Production / work-in-progress (WIP)
- Finished goods
- Available stock
- Stock used
- Stock sold
- Damaged/rejected items
- Returned items
- Dispatched items

Key requirement: movement history, not just a number

The client explicitly wants a **reliable history of inventory movements** — every addition, removal, transfer, and adjustment should be logged with who/what/when — rather than the system simply overwriting a single stock count. This is a foundational architecture decision: inventory should be event-sourced or at minimum fully audit-logged, not just a mutable quantity field.

Data/entities implied

- `InventoryItem` (product/material id, current quantity)
- `InventoryMovement` (item id, type: purchase | production | sale | return | damage | dispatch | adjustment, quantity delta, actor, timestamp, reference — links to `19-Approvals-Audit-Trail.md`)

12. Accounting & Finance

Purpose

The platform is intended to **replace the current manual accounting process** entirely.

Areas requiring visibility

- Sales
- Expenses
- Purchases
- Supplier payments (in aggregate — see scope note in `13-Suppliers.md`)
- Production costs
- Cost of goods (COGS)
- Payroll
- Taxes
- Profit
- Loss
- Financial reports

Reporting requirement (from clarification round)

Client wants **all** financial report types available: income, expenditure, investment, loss, and profit reports. Build a flexible reporting module rather than a fixed small set of reports.

Credit policy

Customers are **not** expected to owe the business — no customer credit terms to build for on the retail/wholesale side.

Data/entities implied

- `LedgerEntry` (type: sale | expense | purchase | payroll | tax, amount, category, date, reference)
- `FinancialReport` (type, period, generated data)

13. Suppliers

Scope change confirmed by client during review. The client does not have a stable material supplier and sources materials from multiple suppliers interchangeably. She explicitly stated: “We don’t have a stable supplier. We source from multiple suppliers, so their info does not need to be in the system. We just need to record as listed in the ‘bulleted’ ones.”

This means: **no dedicated Supplier entity, supplier profile, or supplier payment-history module should be built for MVP.**

What to build instead

Only the material-level tracking already described in `03-Raw-Materials.md`: - Materials purchased - Purchase costs - (Optional free-text note per purchase if the business wants to jot down who they bought from, but this is not a structured/required field)

Note for the team

The original discovery document proposed a full supplier profile/history/outstanding-obligations module. That proposal is now **descoped** based on the client’s clarification. Do not build supplier relationship management for MVP.

14. Logistics & Delivery

Delivery methods in use

- Dispatch riders (local delivery)
- Logistics companies
- Transport companies
- Interstate/city-to-city delivery
- International delivery

Pricing

- **Delivery cost is calculated by weight and location.**
- Some destinations may require multiple delivery legs — the system should be able to represent a delivery as more than one stage/leg where needed.

Integration priority

- **First logistics company to integrate: GIGL.**

Requirements

- Record and display delivery status and tracking information per order.

Data/entities implied

- `DeliveryLeg` (order id, carrier, leg_number, status, tracking_ref)
- `DeliveryPricing` (weight, destination/zone, calculated cost)

15. Marketing & Sales Growth

Business objective

Increasing sales is the client’s major business objective, so the platform should help the business understand **where sales are coming from**, in addition to running marketing activity.

Capabilities to support

- Marketing campaigns
- Promotions
- Sales-channel tracking (attribute each sale to its source/channel)
- Customer recommendations
- Loyalty programs
- Product visibility/boosting
- Sales performance analysis

Data/entities implied

- SaleChannel (order id, channel/source)
- Campaign (name, type, start/end date, linked promotions)

16. Analytics & Management Dashboard

Goal

Management should see the important business information from one place, without needing to be at the factory.

Dashboard data points

- Total sales
- Revenue
- Profit/loss
- Best-selling products
- Slow-moving products
- Low-stock products
- Production status
- Inventory status
- Product profitability
- Sales by channel
- Marketing/source performance
- Partner/investor information (where permitted — see `17-Partners-Investors.md` for access boundaries)

Data/entities implied

This is primarily a reporting/aggregation layer over the entities defined in `02`, `03`, `04`, `08`, `11`, `12`, `14`, `15`. No new core entities, but plan for a dashboard/reporting service that can query across all of them.

17. Partners / Investors

Important clarification

“Partners” in this platform means **business partners/investors** — people who have invested money in the business — **not affiliate marketers or referral partners**. This distinction affects the entire partner module design (see 23-Future-Vision.md for the explicit statement that an affiliate/commission model is *not* part of the initial direction).

Their role and access

- They invest money in the business.
- They receive **quarterly profit sharing**.
- They should have transparency around business performance.
- They should see appropriate financial/account information.
- They should have visibility into inventory.
- They should understand how the business is performing and how their investment is being used.
- They should **not** have access to the retail/wholesale customer-facing interfaces.

Confirmed dashboard flow

Partner/Investor → Login → Business Overview → Investment Information → Performance → Inventory Visibility → Accounts/Reports → Profit Sharing

Profit-sharing model (confirmed in clarification round)

- Profit split: **40% reinvestment / 40% dividends (owners + investors) / 20% cash reserve**.
- Total shares: **1,000,000**.
- Share allocation: **60% to founder/CEO, 40% to partners**.
- Each partner is paid **according to their equity percentage** of that 40%.

Data/entities implied

- Partner (name, equity_percentage, invested_amount)
- ProfitDistribution (period, total_profit, reinvestment_amount, dividend_pool, reserve_amount, per_partner_breakdown)

Priority note

Marked “**Important Later**” in the feature-priority table, not MVP — see 24-MVP-Priorities.md . Given how fully specified the profit-sharing model already is, this may be worth revisiting for earlier scheduling — flagged in 25-Remaining-Clarification-Questions.md .

18. Staff & Access Control

Scale

Approximately **10 people** are expected to use the internal system.

Principle

Not everyone should have access to everything — the system must support **roles and permissions**.

Confirmed roles (illustrative, not necessarily exhaustive)

- Business Owner/Admin
- Sales
- Inventory
- Production
- Finance/Accounting
- Management
- Partner/Investor
- Customer

Requirement

Sensitive activities should be restricted by role, and important actions should be recorded (see `19-Approvals-Audit-Trail.md`).

Data/entities implied

- `User` (name, role, permissions)
- `Role` (name, list of permitted actions)

19. Approvals & Audit Trail

Requirement

Some important actions must require **management approval** before they take effect, and the system must keep a history of: - Who performed an action - What was changed - When it was changed - What was approved/rejected (where applicable)

Actions confirmed to require approval/authorization

- Purchasing
- Production
- Changing of product price
- Moving of funds
- Any employee action to manufacture, alter, remove, transfer, give away, or dispose of materials or finished products **without authorization** (this is a strict policy — the system should make unauthorized versions of these actions impossible, not just logged)

Data/entities implied

- ApprovalRequest (action_type, requested_by, status, approved_by, decided_at)
- AuditLogEntry (actor, action, before_state, after_state, timestamp)

20. Communication

Principle

Communication should be built into the customer experience, especially around orders and delivery.

Channels

- In-platform messages
- SMS
- WhatsApp
- Live chat

Data/entities implied

- Notification (recipient, channel, type, related order id, sent_at)

21. Integrations

Confirmed/identified integration points

- **Paystack** (online payments)
- **WhatsApp**
- **Live chat**
- **SMS**
- **Logistics providers** — first: **GIGL** (see 14-Logistics.md)
- **Marketing/social channels**
- **Future POS/mall sales systems** (see 22-POS-Mall-Sales-System.md)

Data migration

The business wants to **start fresh** rather than migrate old manual records into the new system. No migration tooling required for MVP.

22. POS / Mall Sales System

Vision

The client wants a physical point-of-sale system similar to what a mall/retail store uses, connected to the central platform, so in-person sales stay in sync with the rest of the business.

Expected connections

- Inventory
- Products
- Orders
- Payments
- Customer information
- Accounting
- Analytics

Status

Exact POS requirements are **intentionally deferred** to the next planning stage — do not attempt to fully spec POS screens/hardware now.

Priority conflict — needs resolution

The client’s answer directly on this section marked it “**LATER**”, but the separate feature-priority table (Section 28 of the review document) marked “POS/physical sales” as “**Must Have Now.**” These two answers contradict each other. See `25-Remaining-Clarification-Questions.md` — this should be confirmed with the client before sprint planning locks in scope.

23. Future Vision

Explicitly NOT part of the initial platform direction

- Multiple factories.
- A marketplace connecting unrelated third-party sellers.
- An affiliate/commission model disguised as the partner system — the partner system is specifically for business partners/investors (see [17-Partners-Investors.md](#)).

Longer-term direction (post-MVP)

- Multiple physical stores/locations.
- More distribution locations.
- A large online clothing store.
- Wider customer reach.
- More automation.
- More logistics integration.
- Future POS/mall system integration (full spec, beyond MVP connection points).
- More sophisticated sales and business analytics.

The single most important outcome (guiding principle)

“Customers can shop without struggling, while the owner can see what is happening in the business without needing to be physically present at the factory.”

This should be treated as one of the main design principles guiding the whole product, and used as a north star when prioritizing between competing features.

24. MVP Priorities

Based on the client’s feature-priority table (Section 28 of the review document), classified as follows:

Must Have Now

- Manufacturing
- Production tracking
- Raw materials
- Inventory
- Product catalogue
- Retail e-commerce
- Wholesale
- Payments
- Order tracking
- Returns
- Accounting
- Logistics
- Marketing
- Analytics
- Staff management
- Approvals/audit trail
- WhatsApp/live chat
- POS/physical sales (*see contradiction note below*)

Important Later

- Custom/pre-order requests
- Partner/investor portal
- SMS/notifications

⚠ Contradiction to resolve before sprint planning

The feature-priority table marks “**POS/physical sales**” as **Must Have Now**, but the client’s direct confirmation on Section 24 (POS / Mall Sales System) marked it “**LATER.**” Recommend treating POS as **Later** for MVP scope (deferred spec, per her explicit written answer), and getting written confirmation from her which one is correct before committing sprint capacity either way. See `25-Remaining-Clarification-Questions.md`.

Suggested MVP build order (proposed, pending client sign-off)

1. Core data foundation: Products, Inventory, Users/Roles
2. Manufacturing & Production tracking + Raw materials
3. Retail e-commerce + Payments (Paystack) + Order tracking
4. Wholesale
5. Accounting
6. Returns + Approvals/audit trail
7. Logistics
8. Analytics dashboard
9. Marketing tools
10. WhatsApp/live chat
11. (Deferred) Custom/pre-order, Partner/investor portal, SMS, POS

25. Remaining Clarification Questions

These should ideally be resolved with the client before or very early in Sprint 0. None of them block starting the data model, but they do affect scope/prioritization decisions.

1. **POS/physical sales priority contradiction.** Section 24 direct answer says “LATER”; the feature-priority table says “Must Have Now.” Which is correct?
2. **Wholesale price tiers.** She confirmed multiple wholesale price tiers exist, but the criteria for assigning a customer to a tier (order volume? relationship length? negotiated manually?) hasn’t been defined.
3. **Sample cost in custom orders.** Is the cost of producing the sample included in the quotation the buyer already paid, or billed separately?
4. **Review moderation.** For the newly-added product review feature (retail + wholesale) — should reviews be public immediately, or moderated/approved before showing publicly?
5. **Partner/investor portal timing.** It’s marked “Important Later” in the feature table, but the profit-sharing model and dashboard flow are already fully specified. Worth confirming whether this should be pulled earlier given how ready the spec already is.
6. **Currency.** Assumed to be Nigerian Naira (₦) throughout — please confirm, especially since international delivery and possibly international customers were mentioned.
7. **Data protection.** Given customer personal data (name, phone, location, order history) will be stored, confirm whether NDPR (Nigeria Data Protection Regulation) compliance requirements apply and need to be built in from day one.

26. Final Approval / Sign-off

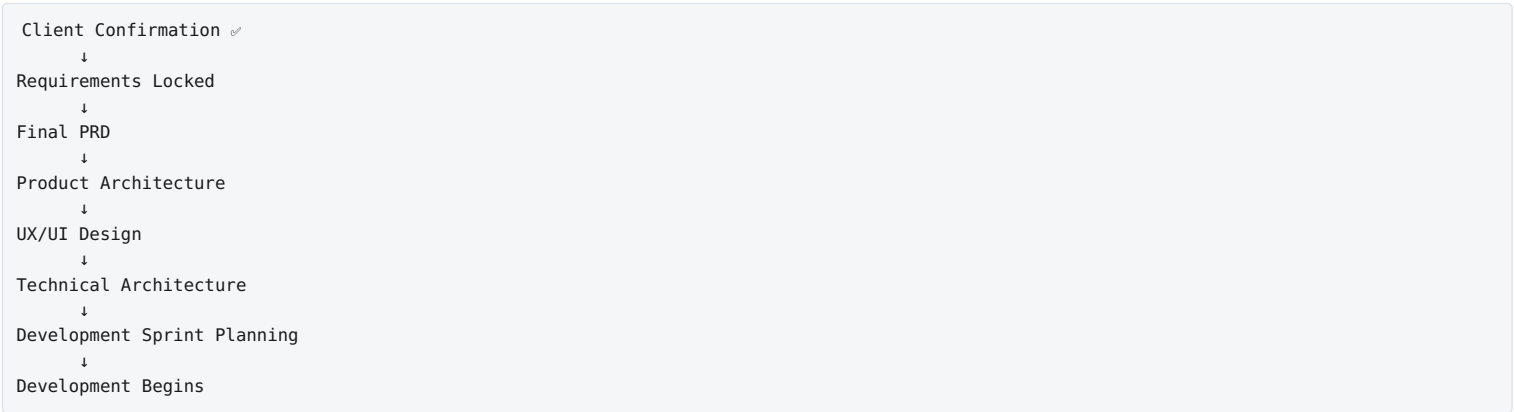
Client confirmed all of the following (Section 30 of the review document):

- ☒ A. Business Understanding — confirmed correct
- ☒ B. Product Direction — confirmed reflects what she wants to build
- ☒ C. Features — reviewed and prioritized (see 24-MVP-Priorities.md)
- ☒ D. Business Rules — workflows and rules reviewed, corrections identified (incorporated throughout files 01-23)
- ☒ E. Missing Information — corrections/additions provided
- ☒ F. Proceed — **approved to proceed to the detailed PRD / product design stage**

Signed

- **Client/Business Name:** SEENTAIR LIMITED
- **Client Name:** Okereke Ndubuka Cynthia
- **Date:** 9th day of September, 2026
- **Signature/Confirmation:** Cynthia

What happens next



This sign-off is the authorization to proceed with development planning.

15. Design Specification

Seentair Limited — Streetwear Commerce Experience

Status: Active design direction **Revision:** 15 September 2026

This section translates the newly confirmed product-design direction into a practical specification for Google Stitch, UI design, and frontend implementation.

The public website should feel like a fashion brand first and a software system second.

15.1 Design Direction

Seentair is a streetwear and clothing brand with a manufacturing operation behind the commerce experience. The public website should therefore lead with fashion, product, movement, and strong visual storytelling rather than a conventional corporate or SaaS hero.

The landing page is a scroll-driven cinematic experience. Scrolling controls the narrative and progressively completes a streetwear outfit.

Core narrative

Empty mannequin → shirt → joggers → sneakers → complete look → shop the look.

The landing page is the entry point to the retail storefront and must naturally transition the visitor into product discovery and shopping.

What is deliberately not locked here

This specification defines the experience, not a premature implementation mechanism. The engineering team can select the most reliable animation technique after evaluating performance, accessibility, asset weight, browser support, and maintainability.

15.2 Scroll-Driven Landing Narrative

Stage	Scroll state	Visual behaviour	Purpose
1	Introduction	Premium minimal fashion scene. Empty mannequin/model is the focal point.	Establish brand and intrigue.
2	Scroll 1	A Seentair shirt transitions onto the mannequin.	Introduce the first product piece.
3	Scroll 2	Joggers are added and the composition shifts toward the full silhouette.	Build the outfit progressively.
4	Scroll 3	Sneakers appear and complete the styling.	Reach the strongest visual moment.
5	Reveal	Complete outfit remains visible with a clear Shop the Look CTA.	Convert visual interest into shopping.

Motion principles

- Scroll controls progress.
- Motion is physical and fashion-editorial.
- Avoid decorative motion that competes with the outfit.
- Do not make shopping dependent on the animation.

15.3 Mobile, Accessibility & Performance

The approved UX/UI requirements already establish a mobile-first customer experience and call for resilience on common Nigerian mobile network conditions. The cinematic landing page must respect those constraints.

Mobile requirements

- Preserve the narrative while simplifying effects where needed.
- Prioritize fast loading over visual complexity.
- Compress and lazy-load heavy visual assets.
- Keep the primary shopping CTA obvious and reachable.
- Provide a reduced-motion/static alternative.
- Never block catalogue access if animation assets fail to load.

Accessibility requirements

The visual experience must coexist with readable typography, clear contrast, keyboard/focus behaviour where applicable, and a prefers-reduced-motion path. Animation is enhancement, not a prerequisite for understanding or purchasing.

Performance rule

A visually impressive landing page that delays product discovery is a failed implementation. The performance budget should be treated as part of the design acceptance criteria.

15.4 Storefront Structure

The cinematic landing page is one part of the broader retail storefront.

Customer journey

*Home / Landing → Shop / Product Discovery → Product Detail → Cart → Checkout
→ Order Confirmation → Order Tracking → Review / Return.*

Primary storefront screens

- Product discovery / browse & search
- Product detail with size and colour selection
- Cart & Paystack checkout
- Order confirmation
- Order tracking
- Post-delivery review submission
- Return request form

Separate application experiences

The authenticated wholesale portal and internal admin/operations dashboard are distinct application surfaces. They should not inherit the cinematic storefront treatment wholesale. They should instead optimize for task completion, clarity, role-appropriate information, and operational efficiency.

Platform boundary

All user-interaction frontends are Angular. They connect to the same Node.js + NestJS Core API and central PostgreSQL database.

15.5 Google Stitch Direction

Use the following as the starting design brief for Google Stitch:

Design a premium, modern streetwear fashion e-commerce website for Seentair Limited, a clothing manufacturer and streetwear brand. The landing page must be cinematic and scroll-driven. Start with a minimal premium fashion scene containing an empty mannequin/model. As the user scrolls, progressively dress the mannequin: first a Seentair shirt, then joggers, then sneakers. Each scroll stage should feel like a physical fashion styling sequence, not a slideshow. After the complete outfit is revealed, show a strong "Shop the Look" CTA leading into the retail catalogue.

Use an editorial fashion aesthetic: bold typography, generous whitespace, large product imagery, restrained interface chrome, premium composition, and confident streetwear energy. Avoid generic SaaS styling, crowded layouts, excessive gradients, and unnecessary decoration.

Design the rest of the storefront around product discovery, collections, product detail, cart, checkout, order confirmation, order tracking, reviews, and returns. Make the entire customer experience mobile-first and performant on constrained mobile networks. Provide clear navigation to the shop and other site areas. The cinematic animation must have a reduced-motion/static fallback and must never prevent the user from shopping.

15.6 Design Acceptance Criteria

The landing page is ready for implementation handoff when all of the following are true:

1. A first-time visitor immediately understands that Seentair is a clothing/streetwear brand.
2. The mannequin-to-complete-outfit story is understandable without instructions.
3. Scroll position clearly controls visual progression.
4. The complete outfit naturally leads to shopping.
5. The page remains usable on mobile.
6. The page remains usable with reduced motion.
7. Visual quality feels premium without compromising load performance.
8. The storefront remains visually distinct from the internal admin application.

15.7 Confirmed Engineering Stack

User-interaction frontends: Angular — Retail Storefront, Wholesale Portal, Admin / Operations Dashboard.

Backend: Node.js + NestJS.

Database: PostgreSQL.

Supporting architecture retained from the existing technical specification includes S3-compatible object storage, Paystack, GIGL through a pluggable logistics adapter, notification providers, and background jobs such as BullMQ/Redis where required.

This design section is intentionally experience-led. The business requirements, role matrix, API specification, data model, QA strategy, operations guide, and delivery phases remain part of the full project documentation.